

SL-SPV Experimental Protocol & Reporting Guide

A reproducible end-to-end protocol that exercises every shipped feature (BUGFIX-001..026, FEATURE-001..011) on a 100-speaker Sinhala+Tamil corpus and produces thesis-grade and journal-grade speaker-verification results.

Last updated: 2026-05-22 · **Code state:** all BUGFIX-NNN and FEATURE-NNN landed up to FEATURE-011 (corpus prep) and FEATURE-010 (PLDA scoring).

Contents

- [Part 1 — Setup](#)

1. Prerequisites & environment
2. Hardware allocation (your 3 machines)

- [Part 2 — Corpus](#)

1. Directory layout
2. Generating list, lookup and cohort files

- [Part 3 — The protocol](#)

1. P0 — Baseline (ECAPA-TDNN, from scratch)
2. P1 — Cross-lingual fine-tune from an English checkpoint
3. P2 — Full feature stack
4. Ablation matrix

5. Multi-seed statistical reporting

- [Part 4 — Outputs & runtime](#)

1. Output artefact inventory
2. Master script template
3. Expected runtime per machine

- [Part 5 — Writing it up](#)

1. Methods section template (LaTeX)
2. Results tables (LaTeX templates)
3. Reproducibility checklist
4. Citation block

- [Appendices](#)

A. Feature-by-feature flag reference B. Troubleshooting C. Expected EER ranges (sanity-check guidance)

Part 1 — Setup

1. Prerequisites & environment

```
# Python 3.10 or newer; tested on 3.11
python -m venv .venv
source .venv/bin/activate
```

```

pip install -r requirements.txt
# transformers >=4.30,<5 is required for FEATURE-001 SSL paths
pip install 'transformers>=4.30,<5'

```

Verify the install:

```

python -c "import torch; print('torch', torch.__version__, 'cuda', torch.cuda.is_available(), 'devices', torch.cuda.device_count())"
python -c "from models.ECAPA_TDNN import MainModel; print(MainModel(nOut=192))" | head -10

```

Set up the portable data root (BUGFIX-013):

```

cp paths.env.example paths.env
# edit paths.env: SL_SPV_DATA_ROOT=/abs/path/to/sl_celeb
source paths.env
echo "$SL_SPV_DATA_ROOT"

```

2. Hardware allocation (your 3 machines)

You have **2× T4 single-machine**, **1× A10**, **1× A40**. Use them like this:

Machine	VRAM	Best workload	Why
A40 (48GB)	48 GB	FEATURE-001 SSL configs (WavLM, XLS-R, mHuBERT)	Headroom for 95M–300M param encoders at batch_size=32+
2× T4 (16 GB ×2 = distributed)	32 GB	ECAPA-TDNN baseline, ablations	Distributed-data-parallel speeds these up ~1.8× on 2 GPUs; mixed precision (already on) covers Turing fp16
A10 (24 GB)	24 GB	Multi-seed reproducibility runs, ECAPA-Small, debugging	Single GPU good for repeated runs that need no DDP coordination

CUDA / cuDNN compatibility note: PyTorch ≥ 2.1 is required (the BUGFIX trail uses `torch.amp.autocast('cuda', ...)`, `inference_mode`, deterministic algorithms). Verify with the `python -c "import torch ..."` line above.

Part 2 — Corpus

3. Directory layout

The corpus-prep script (`tools/sl_dataprep.py`, FEATURE-011) expects this exact tree:

```

$SL_SPV_DATA_ROOT/sl_celeb/
├── si/                                     # Sinhala
│   ├── si_spk000/
│   │   ├── utt00.wav                     # 16 kHz mono, ≥2 s, ≥4 utts per speaker
│   │   ├── utt01.wav
│   │   └── ...
│   └── si_spk059/
│       └── ...
├── ta/                                     # Tamil
│   ├── ta_spk000/ ...
│   └── ta_spk039/ ...
├── en/                                     # Optional: English utterances of the same speakers (for cross-lingual tr
│   └── ...

```

└─ mix/ └─ ...	# Optional: code-switched utterances
-------------------	--------------------------------------

Constraints (enforced at prep time):

- *.wav files only. 16 kHz mono (resample with sox or the loader's auto-resampler from BUGFIX-005).
- Each speaker directory ≥ 4 utterances (the prep script holds out 20 % for test; ≤ 4 leaves <1 train utterance).
- Speaker IDs are arbitrary strings; the prep script assigns contiguous integer labels.
- A speaker present in BOTH si/spk_X and ta/spk_X is treated as the SAME speaker (one integer label). This unlocks cross-lingual trials.

Side resources also under \$SL_SPV_DATA_ROOT:

\$SL_SPV_DATA_ROOT/ └─ sl_celeb/ └─ musan/ └─ RIRS_NOISES/simulated_rirs/	# main corpus (above) # MUSAN noise/music/speech (BUGFIX-016) # reverb (BUGFIX-016)
--	---

4. Generating list, lookup and cohort files

Run the corpus prep script **once**:

python tools/sl_dataprep.py \
--corpus_root \$SL_SPV_DATA_ROOT/sl_celeb \
--out_dir \$SL_SPV_DATA_ROOT/sl_celeb/lists \
--langs si ta \
--test_frac 0.2 \
--target_pairs_per_lang 1000 \
--impostor_ratio 1 \
--cohort_size 500 \
--seed 42

This produces **9 files** in \$SL_SPV_DATA_ROOT/sl_celeb/lists/:

File	Used by	Format
train_list.txt	trainer (--train_list)	<spk_int> <relpath>
test_list.txt	trainer pooled-eval (--test_list)	<label> <enrol> <test>
test_list_si.txt, _ta.txt, _cs.txt	FEATURE-003 per-language eval	same as test_list.txt
spk_lang_lookup.txt	FEATURE-007 (--lang_aux_label_file) and FEATURE-008 (--dann_lang_label_file)	<spk_int> <lang_int>
asnorm_cohort.txt	FEATURE-002 (--as_norm_cohort_list)	one wav path per line
plda_train_list.txt	FEATURE-010 (--plda_train_list)	<spk_int> <relpath>
speakers.csv	reproducibility metadata	spk_label, original_id, lang, n_train, n_test

The bottom of the prep script's output prints nClasses for trainer: --nClasses <N> — paste that value into all configs below.

Part 3 — The protocol

The protocol consists of **three primary configurations** (P0 / P1 / P2), then a **one-feature-at-a-time ablation** stripping features back from P2. Every primary config is run with **3 random seeds** (42 / 123 / 7) and reported as mean $\pm$ std. The ablations are run with seed 42 only (per-feature contribution, not absolute number).

All commands assume the cwd is the repo root and \$SL_SPV_DATA_ROOT is set.

5. P0 — Baseline: ECAPA-TDNN from scratch

```
NCLASSES=$(awk -F, 'END{print NR-1}' $SL_SPV_DATA_ROOT/sl_celeb/lists/speakers.csv)

python trainSpeakerNet.py \
  --config configs/ecapa_tdnns.yaml \
  --nClasses $NCLASSES \
  --train_list $SL_SPV_DATA_ROOT/sl_celeb/lists/train_list.txt \
  --test_list $SL_SPV_DATA_ROOT/sl_celeb/lists/test_list.txt \
  --train_path $SL_SPV_DATA_ROOT/sl_celeb \
  --test_path $SL_SPV_DATA_ROOT/sl_celeb \
  --musan_path $SL_SPV_DATA_ROOT/musan \
  --rir_path $SL_SPV_DATA_ROOT/RIRS_NOISES/simulated_rirs \
  --save_path exps/P0_ecapa_baseline_seed42 \
  --seed 42 \
  --deterministic \
  --max_epoch 80
```

What this tests: can a modern SV architecture learn anything from 100 SL speakers without prior knowledge? Expected: 8-20 % EER. This is your floor.

Hardware: 2×T4 (DDP) or A10. Add --distributed on T4 to use both. Expect ~12-18 h on T4 pair, ~10 h on A10.

6. P1 — Cross-lingual fine-tune from an English checkpoint

This requires an English-pretrained checkpoint (e.g. VoxCeleb1 ECAPA). If you don't have one yet, train one first using the same config but with --train_list pointing at VoxCeleb1 dev.

```
python trainSpeakerNet.py \
  --config configs/ecapa_tdnns.yaml \
  --nClasses $NCLASSES \
  --train_list ... --test_list ... --train_path ... --test_path ... \
  --musan_path ... --rir_path ... \
  --save_path exps/P1_ecapa_finetune_seed42 \
  --seed 42 \
  --deterministic \
  --max_epoch 60 \
  --initial_model $SL_SPV_DATA_ROOT/checkpoints/ecapa_voxceleb1.model \
  --finetune \
  --finetune_freeze frontend \
  --finetune_lr_multiplier 0.1 \
  --llrd --llrd_layer_pattern ecapa --llrd_decay 0.9
```

What this tests: does English-pretrained knowledge transfer to SL? FEATURE-004 freezes the front-end (preserves English acoustic features); FEATURE-005 applies layer-wise LR decay. Expected: 30-50 % relative EER drop vs P0.

Hardware: A40 or A10. Skip DDP — fine-tune runs are shorter (~6 h). Run all 3 seeds in series on one machine.

7. P2 — Full feature stack

Every flag turned on. This is the headline number for the paper.

Create configs/sl_full_stack.yaml:

```
# Inherits from ecapa_tdn.yaml semantics; override the SL-specific paths
model: ECAPA_TDNN
nOut: 192
channels: 1024
n_mels: 80
sample_rate: 16000
log_input: true
encoder_type: ASP

augment: true
batch_size: 200
max_frames: 200
eval_frames: 0
nDataLoaderThread: 5
max_seg_per_spk: 500
seed: 42

test_interval: 1
trainfunc: aamsoftmax
patience: 15
max_epoch: 60

optimizer: adam
scheduler: steplr
lr: 0.001
lr_decay: 0.97
weight_decay: 2e-5

margin: 0.2
scale: 30
nPerSpeaker: 1

initial_model: ${SL_SPV_DATA_ROOT}/checkpoints/ecapa_voxceleb1.model

train_list: ${SL_SPV_DATA_ROOT}/sl_celeb/lists/train_list.txt
test_list:  ${SL_SPV_DATA_ROOT}/sl_celeb/lists/test_list.txt
train_path: ${SL_SPV_DATA_ROOT}/sl_celeb
test_path:  ${SL_SPV_DATA_ROOT}/sl_celeb
musan_path: ${SL_SPV_DATA_ROOT}/musan
rir_path:   ${SL_SPV_DATA_ROOT}/RIRS_NOISES/simulated_rirs

mixedprec: true
prefetch_factor: 2
persistent_workers: true

# ----- FEATURE-004 cross-lingual fine-tune -----
finetune: true
finetune_freeze: [frontend]
finetune_lr_multiplier: 0.1

# ----- FEATURE-005 layer-wise LR decay -----
llrd: true
llrd_decay: 0.9
llrd_layer_pattern: ecapa

# ----- FEATURE-007 language-ID multi-task head -----
lang_aux: true
lang_aux_weight: 0.3
lang_aux_num_classes: 4
lang_aux_label_file: ${SL_SPV_DATA_ROOT}/sl_celeb/lists/spk_lang_lookup.txt
```

```
# ----- FEATURE-003 per-language evaluation -----
per_lang_test_lists:
  si: ${SL_SPV_DATA_ROOT}/sl_celeb/lists/test_list_si.txt
  ta: ${SL_SPV_DATA_ROOT}/sl_celeb/lists/test_list_ta.txt
  cs: ${SL_SPV_DATA_ROOT}/sl_celeb/lists/test_list_cs.txt

# ----- FEATURE-002 AS-Norm -----
as_norm: true
as_norm_cohort_list: ${SL_SPV_DATA_ROOT}/sl_celeb/lists/asnorm_cohort.txt
as_norm_cohort_path: ${SL_SPV_DATA_ROOT}/sl_celeb
as_norm_top_k: 300

# ----- FEATURE-010 PLDA scoring -----
plda: true
plda_train_list: ${SL_SPV_DATA_ROOT}/sl_celeb/lists/plda_train_list.txt
plda_train_path: ${SL_SPV_DATA_ROOT}/sl_celeb
plda_dim: 200

save_path: exps/P2_full_stack_seed42

deterministic: true
distributed: false
port: "8888"
```

Run:

```
python trainSpeakerNet.py --config configs/sl_full_stack.yaml --seed 42 --nClasses $NCLASSES
```

What this tests: every shipped lever working together. Expected: 40-60 % relative EER drop vs P0.

Hardware: A40 strongly preferred (PLDA fit + AS-Norm + per-lang eval triple-runs the eval pass; SSL configs additionally need 24+GB).

8. Ablation matrix

Starting from P2, strip ONE feature at a time. Report the EER delta — that's the per-feature contribution.

Tag	What changes vs P2	Tests
P2	(full stack, all flags on)	headline
P2 -finetune	finetune: false, initial_model: ""	importance of cross-lingual transfer (FEATURE-004)
P2 -llrd	llrd: false	importance of layer-wise LR (FEATURE-005)
P2 -lang_aux	lang_aux: false	importance of multi-task lang head (FEATURE-007)
P2 -as_norm	as_norm: false	importance of score normalisation (FEATURE-002)
P2 -plda	plda: false (falls back to cosine)	importance of PLDA backend (FEATURE-010)

(Single-feature ablations only. Pairwise interactions are out of scope for the first paper; mention as future work.)

Each ablation = one run with seed 42. Six ablations + the P2 reference = 7 runs.

```
# Example: -finetune ablation
python trainSpeakerNet.py --config configs/sl_full_stack.yaml --seed 42 --nClasses $NCLASSES \
  --no_finetune --initial_model "" \
  --save_path exps/P2_minus_finetune_seed42
```

(The `--no_finetune` flag negates the `store_true` default. For YAML, set `finetune: false`.)

9. Multi-seed statistical reporting

For the **primary configs only** (P0, P1, P2 — three of them), run **three seeds**: 42, 123, 7. That's 9 runs total for the primary protocol; report mean $\pm$ std.

```
for SEED in 42 123 7; do
    python trainSpeakerNet.py --config configs/sl_full_stack.yaml \
        --seed $SEED \
        --nClasses $NCLASSES \
        --save_path exps/P2_full_stack_seed${SEED}
done
```

For each primary config, after all 3 seeds finish, aggregate via:

```
# A tiny aggregator (drop into tools/ if you want it permanent)
python - <<'PY'
import re, statistics, glob
for cfg in ['P0_ecapa_baseline', 'P1_ecapa_finetune', 'P2_full_stack']:
    eers, dcfs = [], []
    for path in sorted(glob.glob(f'exps/{cfg}_seed*/result/scores.txt')):
        with open(path) as f:
            best = max(re.findall(r'VEER ([\d.]+),.*MinDCF ([\d.]+)', f.read()),
                       key=lambda t: -float(t[0])) # lowest EER
            eers.append(float(best[0])); dcfs.append(float(best[1]))
    if eers:
        print(f'{cfg:30s} EER = {statistics.mean(eers):.3f}  $\pm$  {statistics.stdev(eers):.3f}'
              f' MinDCF = {statistics.mean(dcfs):.4f}  $\pm$  {statistics.stdev(dcfs):.4f}')
PY
```

Bootstrap confidence intervals for the headline EER (the journal-grade extra step): re-sample trial pairs with replacement, recompute EER, repeat 1000 times, take the 2.5%/97.5% quantiles. Drop this into tools/bootstrap_eer.py when you write that script; for the first thesis pass, std-across-seeds is sufficient.

Part 4 — Outputs & runtime

10. Output artefact inventory

Every run produces this under exps/<save_path>/:

```
exps/<save_path>/
├── model/
│   ├── best.model           # lowest VEER checkpoint
│   ├── best.eer             # the value
│   ├── best_threshold.txt   # EER-point threshold
│   ├── model000000010.model # per-epoch checkpoints
│   └── ...
├── result/
│   ├── scores.txt           # per-epoch VEER / MinDCF / RawVEER (FEATURE-002)
│   ├── run<timestamp>.zip    # snapshot of all *.py at training time
│   └── run<timestamp>.cmd     # exact argparse Namespace
├── tb/                      # TensorBoard logs
├── asnorm_cohort.pt          # FEATURE-002 cohort cache (when --as_norm on)
├── plda.pkl                  # FEATURE-010 PLDA cache (when --plda on)
└── eval_feats_tmp/           # FEATURE-018 streaming eval (when --eval_streaming on)
```

For paper-grade reproducibility, the artefacts you need are: **model/best.model** + **result/run<timestamp>.cmd** + **speakers.csv** + **the corpus prep --seed**.

11. Master script template

A single script you can dispatch across your three machines. Save as `tools/run_all_protocol.sh`:

```
#!/usr/bin/env bash
set -euo pipefail
source paths.env
NCLASSES=$(awk -F, 'END{print NR-1}' $SL_SPV_DATA_ROOT/sl_celeb/lists/speakers.csv)
SEEDS=(42 123 7)

# ---- Primary configs × 3 seeds -----
for SEED in "${SEEDS[@]}; do
    python trainSpeakerNet.py --config configs/ecapa_tdnns.yaml \
        --seed $SEED --nClasses $NCLASSES --deterministic \
        --save_path exps/P0_ecapa_baseline_seed${SEED}

    python trainSpeakerNet.py --config configs/ecapa_tdnns.yaml \
        --seed $SEED --nClasses $NCLASSES --deterministic \
        --initial_model $SL_SPV_DATA_ROOT/checkpoints/ecapa_voxceleb1.model \
        --finetune --finetune_freeze frontend --finetune_lr_multiplier 0.1 \
        --llrd --llrd_layer_pattern ecapa --llrd_decay 0.9 \
        --save_path exps/P1_ecapa_finetune_seed${SEED}

    python trainSpeakerNet.py --config configs/sl_full_stack.yaml \
        --seed $SEED --nClasses $NCLASSES \
        --save_path exps/P2_full_stack_seed${SEED}
done

# ---- Ablations (seed 42 only) -----
for ABLATE in finetune llrd lang_aux as_norm plda; do
    python trainSpeakerNet.py --config configs/sl_full_stack.yaml \
        --seed 42 --nClasses $NCLASSES \
        --no_${ABLATE} \
        --save_path exps/P2_minus_${ABLATE}_seed42
done

echo "All protocol runs complete. Aggregate with tools/aggregate_seeds.py."
```

Dispatch suggestion: A40 runs P2 (heaviest); T4-pair runs P0 (DDP across both); A10 runs P1 + the 5 ablations in series. Total wall-clock budget: ~3–5 days for the primary 9 runs; +1.5 days for ablations.

12. Expected runtime per machine

Run	A40	2× T4 (DDP)	A10
P0 ECAPA from scratch (80 epoch)	~6 h	~13 h	~9 h
P1 fine-tune (60 epoch)	~3 h	~7 h	~5 h
P2 full stack (60 epoch) — heaviest (PLDA fit + per-lang × 3 eval)	~4 h	~9 h	~6 h
One ablation (60 epoch)	~3 h	~7 h	~5 h

Numbers assume `batch_size=200`, `max_frames=200`, 80 mel filterbanks, ECAPA-TDNN-Large. Halve runtime by switching to channels: 512 (ECAPA-Small) if compute-bound — EER usually 0.5–1.5 % worse.

Part 5 — Writing it up

13. Methods section template (LaTeX)

Drop this into your thesis Methods chapter / journal paper §3. Replace bracketed values after the runs finish.

Note: Introduction and Related Work chapter templates live separately under [thesis_chapters/](#). See [thesis_chapters/README.md](#) for the file layout and assembly instructions.

```
\section{Methods}
\label{sec:methods}

\subsection{Corpus}
\label{ssec:corpus}
The SL-SPV corpus comprises [N=100] speakers of Sinhala (n=[60])
and Tamil (n=[40]) sourced from [your sources]. All recordings are mono
16-kHz; the median utterance duration is [X]~s. We hold out 20\% of
each speaker's utterances for evaluation (see Section~\ref{ssec:trials});
the remaining 80\% form the training set ([M] utterances).

\subsection{Trial construction}
\label{ssec:trials}
We construct verification trials at three operating points: monolingual
Sinhala (test\_list\_si.txt; 1\{,}000 target + 1\{,}000 impostor pairs),
monolingual Tamil (1\{,}000 + 1\{,}000), and cross-lingual
(enrolment in one language, test in the other; [K] target +
[K] impostor pairs for speakers present in both languages).
Pair sampling is reproducible from a fixed random seed
(\texttt{tools/sl\_dataprep.py --seed 42}).

\subsection{Architecture}
\label{ssec:arch}
The speaker encoder is ECAPA-TDNN \cite{desplanques2020ecapa} with
 $C=1024$  channels, 80 log-mel filterbanks, embedding dimension 192,
and channel-attentive statistical pooling. The training loss is
additive-angular-margin softmax (AAM-Softmax) \cite{deng2019arcface}
with margin 0.2 and scale 30. We use the open-source
voxceleb\_trainer codebase \cite{slspv2026repo} extended with the
features described below.

\subsection{Cross-lingual transfer}
\label{ssec:transfer}
We initialise from an ECAPA-TDNN checkpoint trained on VoxCeleb1
\cite{nagrani2017voxceleb} ([N=1\{,}251] English speakers,
[N=148\{,}642] utterances). During fine-tuning the mel front-end
(\texttt{torchfb}, \texttt{instancenorm}) is frozen
(FEATURE-004), the learning rate is scaled by 0.1, and layer-wise
LR decay (FEATURE-005) with  $\gamma=0.9$  is applied across the four
ECAPA layers (configured via the \texttt{ecapa} layer-pattern alias).

\subsection{Multi-task language-ID head}
\label{ssec:langaux}
A linear head  $W_{\{\mathrm{aux}\}} \in \mathbb{R}^{4 \times 192}$  predicts
the utterance's primary language (si / ta / en / mix) from the
speaker embedding (FEATURE-007). Total loss:
 $\mathcal{L}_{\{\mathrm{total}\}} = \mathcal{L}_{\{\mathrm{speaker}\}} +
\lambda \mathcal{L}_{\{\mathrm{lang}\}}$ , with  $\lambda=0.3$ .

\subsection{Score backends}
\label{ssec:scoring}
We compare three eval-time scoring backends: (i) cosine distance on
length-normalised embeddings; (ii) two-covariance PLDA
\cite{sizov2014unifying} (FEATURE-010,  $d_{\{\mathrm{LDA}\}}=200$ );
```

and (iii) adaptive symmetric score normalisation (AS-Norm) \cite{matejka2017analysis} (FEATURE-002, top- $k=300$, cohort of 500 in-domain impostors) on top of either backend.

\subsection{Per-language evaluation}
\label{ssec:perlang}
EER and MinDCF \cite{nist2008sre} are reported per language and as a pooled cross-trial number (FEATURE-003).

\subsection{Reproducibility}
\label{ssec:repro}
All experiments use the \texttt{--deterministic} switch (BUGFIX-017), which sets the cuDNN deterministic flag, disables TF32, and enables \texttt{torch.use_deterministic_algorithms}. Primary configurations are run with three random seeds (42, 123, 7); we report $\mathrm{mean} \pm \mathrm{std}$.

14. Results tables (LaTeX templates)

Headline table

```
\begin{table}[t]
\centering
\caption{Speaker-verification performance on the SL-SPV evaluation set.
Lower is better. Pooled = concatenation of Sinhala, Tamil, and
cross-lingual trials. Numbers are  $\mathrm{mean} \pm \mathrm{std}$  over
three random seeds (42, 123, 7).}
\label{tab:headline}
\begin{tabular}{lcccc}
\toprule
\textbf{Config} & \multicolumn{4}{c}{\textbf{EER \% (Pooled / Si / Ta / Cs)}} \\
\midrule
P0 ECAPA-TDNN from scratch & [X.X  $\pm$  Y.Y] & [X.X  $\pm$  Y.Y] & [X.X  $\pm$  Y.Y] & [X.X  $\pm$  Y.Y] \\
P1 + cross-lingual fine-tune & [X.X  $\pm$  Y.Y] & [X.X  $\pm$  Y.Y] & [X.X  $\pm$  Y.Y] & [X.X  $\pm$  Y.Y] \\
P2 full stack (ours) & \textbf{[X.X  $\pm$  Y.Y]} & \textbf{[X.X  $\pm$  Y.Y]} & \textbf{[X.X  $\pm$  Y.Y]} & \textbf{[X.X  $\pm$  Y.Y]} \\
\bottomrule
\end{tabular}
\end{table}
```

Ablation table

```
\begin{table}[t]
\centering
\caption{Single-feature ablations on the SL-SPV evaluation set.
Numbers are pooled EER (\%) with seed 42;  $\Delta$  = increase in EER
when the named feature is removed from the full stack (P2).}
\label{tab:ablation}
\begin{tabular}{lc}
\toprule
\textbf{Ablation} & \textbf{Pooled EER \%} &  $\Delta$  \\
\midrule
P2 (all features) & [X.X] & 0 \\
\midrule
-- FEATURE-004 fine-tune & [X.X] &  $+\Delta$  [Y.Y] \\
-- FEATURE-005 LLRD & [X.X] &  $+\Delta$  [Y.Y] \\
-- FEATURE-007 lang-aux & [X.X] &  $+\Delta$  [Y.Y] \\
-- FEATURE-002 AS-Norm & [X.X] &  $+\Delta$  [Y.Y] \\
-- FEATURE-010 PLDA & [X.X] &  $+\Delta$  [Y.Y] \\
\bottomrule
\end{tabular}
\end{table}
```

15. Reproducibility checklist

Tick every box before submitting. Each maps to something the trainer / docs already provide:

Code is open-source. The repo `voxceleb_trainer` (this fork) is on GitHub at the commit-hash recorded in `result/run*.zip`.

All hyperparameters in YAML. No CLI-only flags in the protocol; every choice in `configs/sl_full_stack.yaml`.

Seeds reported. Three seeds (42, 123, 7) for primary configs; one (42) for ablations. Both `--seed` and `--deterministic` (BUGFIX-017).

Software versions pinned. `requirements.txt` + the `transformers>=4.30,<5` constraint for FEATURE-001. Record `torch.__version__` and cuda toolkit version in the methods section.

Hardware reported. Specify GPUs used (A40 / 2× T4 / A10) per primary config.

Corpus prep deterministic. `tools/sl_dataprep.py --seed 42` produces identical lists.

All artefacts saved. `model/best.model`, `result/run<timestamp>.cmd`, `result/run<timestamp>.zip`, `speakers.csv`, `asnorm_cohort.pt`, `plda.pkl` — every file other practitioners need to reproduce.

Trial pairs published. `test_list_*.txt` files committed to a public dataset card (Zenodo, HuggingFace Datasets).

Ablation included. Single-feature ablation table demonstrates each contribution is real.

Statistical claims supported. Headline numbers reported as $\{\}\mathrm{mean}\{\}\pm\{\}\mathrm{std}$ over 3 seeds.

16. Citation block

Paste this into your `.bib`:

```
@misc{slspv2026repo,
  title = {SL-SPV: a low-resource Sinhala/Tamil speaker-verification toolkit
    built on {voxceleb\_trainer}},
  author = {[Your Name]},
  year = {2026},
  url = {https://github.com/[user]/voxceleb_trainer-SL-SPV},
  note = {Commit [HASH] at time of paper submission.}
}

@inproceedings{desplanques2020ecapa,
  title = {{ECAPA-TDNN}: Emphasized Channel Attention, Propagation and
    Aggregation in {TDNN} Based Speaker Verification},
  author = {Desplanques, Brecht and Thienpondt, Jenthe and Demuynck, Kris},
  booktitle = {Interspeech},
  year = {2020},
}

@inproceedings{deng2019arcface,
  title = {{ArcFace}: Additive Angular Margin Loss for Deep Face Recognition},
  author = {Deng, Jiankang and Guo, Jia and Xue, Niannan and Zafeiriou, Stefanos},
  booktitle = {CVPR},
  year = {2019},
}

@inproceedings{matejka2017analysis,
  title = {Analysis of Score Normalization in Multilingual Speaker Recognition},
  author = {Matejka, Pavel and others},
  booktitle = {Interspeech},
  year = {2017},
}
```

```

@inproceedings{sizov2014unifying,
  title      = {Unifying Probabilistic Linear Discriminant Analysis Variants in
                Biometric Authentication},
  author     = {Sizov, Aleksandr and Lee, Kong Aik and Kinnunen, Tomi},
  booktitle  = {S+SSPR},
  year       = {2014},
}

@inproceedings{nagrani2017voxceleb,
  title      = {VoxCeleb: A Large-Scale Speaker Identification Dataset},
  author     = {Nagrani, Arsha and Chung, Joon Son and Zisserman, Andrew},
  booktitle  = {Interspeech},
  year       = {2017},
}

@inproceedings{ganin2015dann,
  title      = {Unsupervised Domain Adaptation by Backpropagation},
  author     = {Ganin, Yaroslav and Lempitsky, Victor},
  booktitle  = {ICML},
  year       = {2015},
}

@inproceedings{baevski2020wav2vec2,
  title      = {wav2vec 2.0: A Framework for Self-Supervised Learning of Speech
                Representations},
  author     = {Baevski, Alexei and Zhou, Henry and Mohamed, Abdelrahman and Auli, Michael},
  booktitle  = {NeurIPS},
  year       = {2020},
}

@article{chen2022wavlm,
  title      = {{WavLM}: Large-Scale Self-Supervised Pre-Training for Full Stack
                Speech Processing},
  author     = {Chen, Sanyuan and others},
  journal    = {IEEE Journal of Selected Topics in Signal Processing},
  year       = {2022},
}

```

Appendices

Appendix A — Feature-by-feature flag reference

Quick map from analysis-doc item to the trainer flag(s) and the bugfix/feature doc:

Item	Doc	Trainer flags
§3.1 #1 cross-lingual fine-tune	FEATURE-004	<code>--finetune, --finetune_freeze, --finetune_lr_multiplier, --initial_model</code>
§3.1 #1 LLRD	FEATURE-005	<code>--llrd, --llrd_decay, --llrd_layer_pattern</code>
§3.1 #2 learnable language-aware front-end	FEATURE-001	<code>--model SSLFrontendSpeaker, --ssl_encoder_name, --ssl_freeze/--no_ssl_freeze, --ssl_layer</code>
§3.1 #3 AS-Norm	FEATURE-002	<code>--as_norm, --as_norm_cohort_list, --as_norm_cohort_path, --as_norm_top_k</code>
§3.1 #4 per-lang threshold	FEATURE-003	<code>--per_lang_test_lists</code>
§3.2 #6 ECAPA-TDNN	FEATURE-006	<code>--model ECAPA_TDNN, --channels, --n_mels</code>
§3.2 #7 lang-aux head	FEATURE-007	<code>--lang_aux, --lang_aux_weight, --lang_aux_label_file</code>
§3.3 #11 DANN	FEATURE-008	<code>--dann_lang, --dann_channel, --dann_lang_lambda, ...</code>
§3.3 #12 SSL pretraining (deferred)	FEATURE-009	(no code yet — design doc only)
§3.3 #13 PLDA	FEATURE-010	<code>--plda, --plda_train_list, --plda_dim</code>
§4.2 #12 corpus prep	FEATURE-011	<code>tools/sl_dataprep.py</code>

Appendix B — Troubleshooting

Out of memory on T4 (16GB) during ECAPA-TDNN training:

- Drop `batch_size` from 200 → 128.
- Drop channels: 1024 → channels: 512 (ECAPA-Small) for ~3 GB less memory.
- Ensure `mixedprec: true` is set (already in `configs/ecapa_tdn.yaml`).

CUDA error: device-side assert triggered during training with `lang_aux/dann`:

- The `lang_label` in `spk_lang_lookup.txt` is out of range. The lookup loader validates this and raises with a line number — re-check.

PLDA fit needs $\geq 2 \times \text{plda_dim}$ speakers error from `_setup_plda`:

- Reduce `plda_dim`. With 100 speakers, `max plda_dim = 50`. For full PLDA performance, use a larger PLDA-train set (e.g. include English VoxCeleb1 speakers in `plda_train_list.txt`).

Per-language EER for cs is 0% or undefined:

- Cross-lingual trial list is empty because no speaker exists in both languages. Either provide cross-lingual recordings of the same speaker, or remove `cs:` from `per_lang_test_lists` (the trainer will still produce pooled + monolingual numbers).

ssl_freeze: true config not honoured under lang_aux: true:

- These are separate flags; `lang_aux_head` is NOT under `__L__` so the loss-head freeze doesn't apply to it. Use FEATURE-004's `finetune_freeze: [module.lang_aux_head]` to freeze it explicitly.

Reproducibility check failing on T4:

- TF32 must be disabled (`--deterministic` does this). Also disable cuDNN benchmark with `torch.backends.cudnn.benchmark = False` (BUGFIX-017 does it). T4 + Turing should be bit-exact with these flags.

Appendix C — Expected EER ranges (sanity guidance)

These are *order-of-magnitude* expectations for 100-speaker SL data, not promises. If your numbers are way off, something's wrong with the corpus or the pipeline.

Config	Pooled EER expected range	Notes
P0 ECAPA from scratch	8 % - 20 %	Highly dependent on utterance count per speaker. If <50 utts/spk, expect upper end.
P0 ResNetSE34L baseline	12 % - 25 %	ECAPA outperforms ResNetSE34L by 2-4 % EER on this corpus size (\$3.2 #6 estimate).
P1 fine-tune (frontend frozen)	5 % - 12 %	30-50 % relative drop vs P0 if the English checkpoint is reasonable.
P2 full stack	3 % - 8 %	40-60 % relative drop vs P0. AS-Norm + PLDA + lang-aux compound.
P2 with FEATURE-001 (SSL frontend instead of ECAPA mel)	2 % - 6 %	If your encoder is mHuBERT-147 (best multilingual coverage for SL). Higher cost.

Red flags:

- $P0 < 3\%$: probably trial-pair leakage. Re-check the prep --seed and verify enrolment/test utterances don't overlap.
- $P2 \geq P0$: a feature is broken. Single-feature ablation will isolate.
- Per-lang EER asymmetric (si $\gg$ ta or vice-versa): expected if your corpus is unbalanced; *not* a bug. Worth a sentence in the discussion.

End of guide. If you find a section that doesn't match what the trainer actually does at the time of your run, the trainer is right; file an issue and update this guide.